

PYTHON WORKSHOP · PYGAME

Vesmírné Crossy Road

2D top-down arkádová hra v Pythonu

Bc. Stefan Rajilić

Karel Šafr, Ph.D. · 2025/2026

Co je to za hru?

Hra je inspirovaná mobilní hrou **Crossy Road** – hráč musí přejít z jednoho okraje obrazovky na druhý přes pohybující se překážky.

Mechanika je záměrně jednoduchá, ale implementace skrývá zajímavé technické problémy: pohyb po mřížce, plynulý pohyb překážek, kolize, správa stavů...

- ▶ **Téma:** vesmír — loď, meteority, trosky satelitů
- ▶ **Ovládání:** šipky na klávesnici
- ▶ **Cíl:** dostat se ze dna na horní okraj
- ▶ **Jazyk:** Python 3 + pygame
- ▶ **Rozsah:** 1 soubor main.py, ~670 řádků



Herní mechanika

OVLÁDÁNÍ



Šipky – každý stisk = 1 krok v mřížce

ŽIVOTY

3

Po srážce s překážkou -1 život

SKÓRE

+1 / +50

+1 za krok nahoru, +50 za dokončení levelu

GRACE PERIOD

1,5 s

Neporazitelnost + blikání po zásahu

- ▶ **Bezpečný ostrůvek** uprostřed mapy (řádek 7) – bez překážek, místo k odpočinku
- ▶ **Progresivní obtížnost** – každý level je rychlejší a má více překážek
- ▶ **High-score** se ukládá do souboru a přetrvá i po zavření hry

Herní mřížka

- ▶ Okno **800 × 600 px**
- ▶ Buňka mřížky: **TILE = 40 px**
- ▶ Rozměr: **20 sloupců × 15 řádků**
- ▶ Start hráče: **sloupec 10, řádek 14**
- ▶ Cíl: **řádek 0** (horní okraj)

Klíčové konstanty

```
TILE    = 40          # px/buňka
COLS    = 800 // 40   # = 20
ROWS    = 600 // 40   # = 15
START_COL = COLS // 2  # = 10
START_ROW = ROWS - 1   # = 14
```

Proč mřížka?

Pohyb po mřížce zjednodušuje logiku hry — neřeším fyziku, ale jen přechod mezi buňkami. Pixelové souřadnice se vždy přepočítávají z mřížkových, nikdy se neukládají přímo.

Rozdíl od překážek

Hráč skáče **skokově** po buňkách, překážky se pohybují **plynule** v pixelech (float souřadnice).

Překážky a pruhy

Každý aktivní řádek je definován v seznamu **PRUHY** jako čtveřice hodnot:

```
(řádek, rychlost, typ, počet)
# ↑      ↑      ↑      ↑
# index px/sn. 0=meteor výchozí
# mřížky <0 = ← 1=trosky počet

( 1,  2.5, 0, 2), # řádek 1, doprava
( 2, -2.0, 1, 1), # řádek 2, doleva
( 6, -3.5, 1, 1), # řádek 6, rychle
# řádek 7 = OSTRŮVEK (chybí)
( 8, -2.5, 1, 2), ...
```

- ▶ **12 pruhů celkem** – řádky 1–6 a 8–13
- ▶ **Řádek 7** záměrně chybí → bezpečný ostrůvek
- ▶ Typy překážek:
 - ▶ ▶ **typ 0** – meteorit (velký nebo malý, náhodně)
 - ▶ ▶ **typ 1** – trosky satelitu
- ▶ Po opuštění obrazovky se překážka **zacyklí** na opačné straně s náhodným offsetem
- ▶ Rychlost se **násobí speed_mult** za běhu, ne při inicializaci

Stavový automat

SPLASH $\xrightarrow{(5\text{ s nebo klávesa})}$ MENU $\xrightarrow{\hspace{2cm}}$ SETTINGS | | | | ENTER "Hrát hru" ENTER / ESC | | | ▼▼ | PLAYING
 $\xleftarrow{\hspace{2cm}}$ MENU | | | dosažení cíle | | | ▼ | LEVEL_UP $\xrightarrow{(1,5\text{ s animace})}$ PLAYING (nový level) | | | životy = 0 | |
| ▼ | GAMEOVER $\xrightarrow{(\text{ENTER})}$ MENU | $\xrightarrow{(\text{libovolná klávesa/klik})}$ MENU

SPLASH = 0

Úvodní obrazovka, 5 s

MENU = 1

Hlavní nabídka

SETTINGS = 2

Výběr lodi

PLAYING = 3

Aktivní hra

LEVEL_UP = 5

Přechod levelu, 1,5 s

GAMEOVER = 4

Konec hry

Architektura kódu

```
main.py |— Konstanty | |— barvy, rozměry, FPS | |— jména  
souborů assetů | |— PRUHY - definice pruhů překážek | |—  
Inicializace pygame + display + clock | |— nacti_obrazek() -  
loader s fallbackem | |— High-score: nacti_highscore(),  
uloz_highscore() | |— Třídy spritů | |— Lod - hráčova loď  
(mřížka) | |— Prekazka - letící překážky (float) | |— Vybuch  
- animovaný výbuch | |— Parallax hvězdy + pomocné funkce | |—  
kresli_hvezdy() | |— vytvor_prekazky() | |— vykresli_pozadi()  
| |— vykresli_trysky() | |— vykresli_hud() | |— Inicializace  
herních proměnných | |— while running: | |— EVENT | |— UPDATE | |—  
RENDER
```

- ▶ Vše v **jednom souboru** main.py
- ▶ Všechny sprite třídy dědí z **pygame.sprite.Sprite**
- ▶ Skupiny spritů (**Group**) umožňují hromadné update() a draw()
- ▶ Herní smyčka striktně rozdělena na **EVENT / UPDATE / RENDER**
- ▶ Assety se načítají **jednou před smyčkou** (předcházení lagu)
- ▶ Obrázky s fallbackem – hra **funguje i bez assetů** (fialový placeholder)

Třída Lod

```
class Lod(pygame.sprite.Sprite):
    OBRAZKY = [] # sdílené obrázky lodí
    NAHLEDKY = [] # zvětšené náhledy pro menu

    def __init__(self, index_lodi=0):
        pygame.sprite.Sprite.__init__(self)
        self.image = Lod.OBRAZKY[index_lodi]
        self.rect = self.image.get_rect()
        self.grid_col = START_COL # logická pozice
        self.grid_row = START_ROW
        self._sync_rect()

    def _sync_rect(self):
        self.rect.x = self.grid_col * TILE + 3
        self.rect.y = self.grid_row * TILE + 3

    def move(self, d_col, d_row):
        novy_col = max(0, min(COLS-1, self.grid_col + d_col))
        novy_row = max(0, min(ROWS-1, self.grid_row + d_row))
        pohnul_se = (novy_col != self.grid_col or
                     novy_row != self.grid_row)
        self.grid_col = novy_col
        self.grid_row = novy_row
        self._sync_rect()
        return pohnul_se, pohnul_se and self.grid_row == 0
```

- ▶ **Pozice jako mřížka** – grid_col, grid_row, ne pixely
 - Pixely se vždy přepočítávají přes _sync_rect()
- ▶ **Offset +3 px** centruje sprite 36px v buňce 40px
- ▶ **move()** vrací tuple:
 - ▶ ▶ pohnul_se – skutečně se posunul
 - ▶ ▶ dosahl_cile – dorazil na řádek 0
- ▶ **4 variány lodi** – obrázky sdíleny v třídním atributu OBRAZKY (načtou se jednou)

Přepočet souřadnic: mřížka → pixels

VZOREC _SYNC_RECT()

```
rect.x = grid_col * TILE + 3  
rect.y = grid_row * TILE + 3
```

Offset +3 px: sprite má 36 px, buňka 40 px →
2px mezera na každé straně

PŘÍKLAD – LOĎ NA START POZICI

```
grid_col = 10, grid_row = 14  
rect.x = 10 * 40 + 3 = 403 px  
rect.y = 14 * 40 + 3 = 563 px
```

PŘEKÁŽKY – CENTRY

```
rect.centery = radek * TILE  
               + TILE // 2  
# Řádek 3: 3×40+20 = 140 px
```

Překážky jsou vždy přesně uprostřed svého
pruhu

Proč takto? Logická poloha (mřížka) a vizuální poloha (pixels) jsou oddělené. Pokud bych TILE změnil z 40 na 50, stačí změnit jednu konstantu — vše se přepočítá automaticky.

Třída Prekazka

```
class Prekazka(pygame.sprite.Sprite):
    _cache = {} # sdílené textury

    def __init__(self, radek, rychlost, typ):
        ...
        self.pos_x = float(self.rect.x)

    def update(self, speed_mult=1.0):
        self.pos_x += self.rychlost * speed_mult

        if self.rychlost > 0 and \
            self.pos_x > WIDTH + self.rect.width:
            self.pos_x = -self.rect.width \
                - random.randint(10, 130)

        elif self.rychlost < 0 and \
            self.pos_x < -self.rect.width:
            self.pos_x = WIDTH + random.randint(10, 130)

        self.rect.x = int(self.pos_x)
```

- ▶ **Float pos_x** místo int – plynulý pohyb i při rychlosti 1.5 px/sn.
Bez float by 1.5 px/sn. znamenalo střídání 1 a 2 px → trhání
- ▶ **Cache _cache** – slovník s klíčem (název, šířka, výška); stejný obrázek se načítá jen jednou
- ▶ **Náhodný offset** při zácyklení – překážky nejsou synchronizované
- ▶ **speed_mult** se předává jako parametr update() – čistší než globální proměnná

Třída Vybuch – animovaný výbuch

```
class Vybuch(pygame.sprite.Sprite):
    FRAMES = None # lazy loading
    FRAME_TRVANI = 70 # ms / snímek

    def _nacti_frames(cls):
        frames = []
        for i in range(1, 8):
            soubor = f"vybuch_{i:02d}.png"
            if path.exists(soubor):
                surf = pygame.image.load(soubor)
                frames.append(surf)
        if not frames:
            frames = [nacti_obrazek("vybuch.png", ...)]
        cls.FRAMES = frames

    def update(self):
        nyní = pygame.time.get_ticks()
        if nyní - self.cas_snimku ≥ self.FRAME_TRVANI:
            self.snimek += 1
            if self.snimek ≥ len(Vybuch.FRAMES):
                self.kill() # odstraní ze skupiny
                return
            self.image = Vybuch.FRAMES[self.snimek]
```

- ▶ Animace z **7 snímků** – soubory vybuch_01.png ...
vybuch_07.png
- ▶ Každý snímek trvá **70 ms** → celková délka ~490 ms
- ▶ **Lazy loading** – FRAMES se načtou až při prvním výbuchu
(None = ještě nenačteno)
- ▶ Snímky sdíleny mezi všemi instancemi přes **třídní atribut**
- ▶ **Fallback** – pokud animace neexistuje, použije se jeden
obrázek vybuch.png
- ▶ Po skončení animace sprite **kill()** – odstraní se ze všech
skupin automaticky

Herní smyčka – EVENT / UPDATE / RENDER

```
while running:

    # — EVENT —————
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            if stav == PLAYING:
                if event.key == pygame.K_UP:
                    pohyb = (0, -1)

                ...
                lod.move(*pohyb)

    # — UPDATE —————
    par_far_y = (par_far_y + 0.15) % HEIGHT
    par_near_y = (par_near_y + 0.40) % HEIGHT

    if stav == PLAYING:
        prekazky.update(speed_mult)
        vybuch_grp.update()

    # — RENDER —————
    if stav == PLAYING:
        vykresli_pozadi()
        prekazky.draw(screen)
        vykresli_trysky(lod)
        hrac_grp.draw(screen)
        ...
    pygame.display.flip()
    clock.tick(FPS)
```

- ▶ **EVENT** – reaguje na klávesy podle aktuálního stavu
- ▶ **UPDATE** – parallax hvězdy scrollují vždy, i v menu; překážky jen ve stavu PLAYING
- ▶ **RENDER** – pevné pořadí vykreslování (back-to-front)
- ▶ **double-buffer** – flip() zobrazí celý snímek najednou → bez blikání
- ▶ **clock.tick(60)** – omezí smyčku na 60 FPS

Pořadí RENDER (PLAYING):

1. Pozadí + hvězdy
- ▶ 2. Překážky
- ▶ 3. Trysky
- ▶ 4. Loď
- ▶ 5. Výbuchy
- ▶ 6. HUD

Detekce kolizí a grace period

```
# Grace period = neporazitelnost po zásahu
nyni = pygame.time.get_ticks()

if nyni - cas_zasahu > GRACE_PERIOD: # 1500 ms
    zasahy = pygame.sprite.spritecollide(
        lod, prekazky, False
    )
    if zasahy:
        zivoty -= 1
        cas_zasahu = nyni
        vybuch_grp.add(Vybuch(
            lod.rect.centerx,
            lod.rect.centery
        ))
        lod.reset()
    if zivoty <= 0:
        uloz_highscore(skore)
        stav = GAMEOVER

# Blikání lodi při grace period
if nyni - cas_zasahu < GRACE_PERIOD \
and (nyni // 100) % 2 == 1:
    blik = pygame.Surface((TILE, TILE), SRCALPHA)
    blik.fill((255, 255, 255, 90))
    screen.blit(blik, ...)
```

- ▶ **spritecollide()** – pygame porovná recty dvou skupin najednou ($O(n)$)
- ▶ Třetí parametr **False** – překážky se při kolizi neodstraní
- ▶ **Grace period 1 500 ms** – po zásahu není kolize detekována. Hráč dostane čas utéct.
- ▶ Blikání každých **100 ms** – viditelná vizuální zpětná vazba:
 - ▶ ▶ (ticks // 100) % 2 == 1 → bliká
- ▶ Při zásahu se loď **resetuje na start** a zobrazí se animace výbuchu
- ▶ Při 0 životech → **uloz_highscore() + GAMEOVER**

Systém levelů a obtížnosti

Obtížnost se zvyšuje **dvěma způsoby zároveň**:

```
# 1) Rychlost - speed_mult +0.2 per level
self.pos_x += self.rychlost * speed_mult
# Level 1: 2.5 × 1.0 = 2.5 px/sn.
# Level 5: 2.5 × 1.8 = 4.5 px/sn. (+80%)

# 2) Počet překážek - +1 každé 2 levely
def vytvor_prekazky(level=1):
    bonus = (level - 1) // 2
    # level 1→0, 2-3→+1, 4-5→+2, ...
    for radek, rychlost, typ, pocet in PRUHY:
        for _ in range(pocet + bonus):
            skupina.add(Prekazka(...))
```

LEVEL	SPEED_MULT	BONUS / PRUH	CELKEM PX/SN. (PRUH 2.5)
1	1.0	+0	2.5
2	1.2	+0	3.0
3	1.4	+1	3.5
4	1.6	+1	4.0
5	1.8	+2	4.5
6+	2.0+	+2+	5.0+

Při přechodu levelu se celá skupina překážek přegeneruje – nový level, nové náhodné pozice překážek.

Parallax hvězdné pozadí

```
# Dvě vrstvy hvězd generované při startu
HVEZDY_FAR = [(random.randint(0,W),
               random.randint(0,H))
               for _ in range(70)]
HVEZDY_NEAR = [(random.randint(0,W),
                random.randint(0,H))
                for _ in range(30)]

# UPDATE - každý snímek (i v menu!)
par_far_y = (par_far_y + 0.15) % HEIGHT
par_near_y = (par_near_y + 0.40) % HEIGHT

# RENDER - offset Y + zacyklení přes modulo
for x, y in HVEZDY_FAR:
    dy = int((y + par_far_y) % HEIGHT)
    pygame.draw.circle(screen,
                       (165,165,200), (x, dy), 1)

for x, y in HVEZDY_NEAR:
    dy = int((y + par_near_y) % HEIGHT)
    pygame.draw.circle(screen,
                       WHITE, (x, dy), 2)
```

- ▶ **Vzdálená vrstva** – 70 hvězd, šedavé, poloměr 1 px, rychlost 0.15 px/sn.
- ▶ **Blízká vrstva** – 30 hvězd, bílé, poloměr 2 px, rychlost 0.40 px/sn.
 - Blízká = 2.67× rychlejší → pocit hloubky (parallax efekt)
- ▶ **Modulo HEIGHT** – hvězdy se zacyklí přes spodní okraj
- ▶ Scrollují **vždy** – i na splash obrazovce a v menu

▶ Pozice se nepřepisují – přičítá se jen **float offset**
FAR VRSTVA

70 ★

0.15 px/sn., ø1px, šedé

NEAR VRSTVA

30 ★

0.40 px/sn., ø2px, bílé

Vizuální efekty

ANIMOVANÉ TRYSKY LODÍ

```
# 3 snímky střídané každých 80 ms
frame = (pygame.time.get_ticks() // 80) % 3
vel = [10, 7, 13][frame] # výška
sír = [ 7, 5, 9][frame] # šířka
# Vnější elipsa - oranžová
pygame.draw.ellipse(screen, (255,110,10), ...)
# Vnitřní elipsa - žlutobílá
pygame.draw.ellipse(screen, (255,230,80), ...)
```

FLASH OVERLAY PŘI LEVEL-UP

```
elapsed = pygame.time.get_ticks() - level_up_cas
alpha = max(0, int(230 * (1.0 - elapsed / 1500.0)))
overlay = pygame.Surface((W,H), SRCALPHA)
overlay.fill((255,255,255, alpha))
screen.blit(overlay, (0,0))
```

► Trysky pod lodí:

- ► Kreslí se PŘED lodí (za ní vizuálně)
- ► 3 snímky × 80ms = animace plamene

- **Level-up flash:** bílý overlay alpha od 230 → 0 za 1,5 s (lineární fade)
- **Bezpečný ostrůvek** – řádek 7, tmavě zelený, orámovaný zelenými linkami
- **Blikání lodí** při grace period – poloprůhledný bílý čtverec

Všechny efekty jsou nakresleny přímo přes **pygame.draw** nebo **Surface.blit** – žádné externí knihovny.

High-score a persistence dat

```
# Soubor highscore.txt vedle main.py
HS_FILE = path.join(
    path.dirname(__file__),
    "highscore.txt"
)

def nacti_highscore():
    try:
        with open(HS_FILE, 'r') as f:
            return int(f.read().strip())
    except Exception:
        return 0 # soubor neexistuje → 0

def ulozi_highscore(skore):
    if skore > nacti_highscore():
        with open(HS_FILE, 'w') as f:
            f.write(str(skore))
```

- ▶ Rekord se ukládá do **highscore.txt** – jednoduchý textový soubor
- ▶ Zapisuje se jen při **překonání rekordu** – porovnání před zápisem
- ▶ **try/except** – soubor nemusí existovat (první spuštění), hra nekrachne
- ▶ Uložení probíhá při:
 - ▶ ▶ Game Over
 - ▶ ▶ Zavření okna (QUIT)
 - ▶ ▶ Výběr "Ukončit hru" v menu
- ▶ Rekord se zobrazuje v **menu**, v **HUDu** a na obrazovce **Game Over**

SHRNUTÍ

Co hra umí

HERNÍ MECHANIKY

- ✓ Pohyb po mřížce (šipky)
- ✓ 12 pruhů překážek
- ✓ 3 životy + grace period
- ✓ Bezpečný ostrůvek
- ✓ Progresivní levely
- ✓ High-score (soubor)

TECHNICKÉ ŘEŠENÍ

- ✓ `pygame.sprite.Sprite`
dědičnost
- ✓ Float souřadnice překážek
- ✓ Cache textur
- ✓ Stavový automat (6 stavů)
- ✓ Parallax hvězdy (2 vrstvy)
- ✓ Animované výbuchy (7
snímků)